

Class 15: Optimizing APIs with Redis Caching & Mentioning Your Project in Resume

Your API works. It's paginated, searchable, sortable. But every request still hits the database. Learn how **Redis caching** makes APIs 10-100x faster, and how to **describe this project on your resume** so interviewers actually care.

Part 1: Why Caching?

The Problem

Every time a user hits `GET /menu/items/`, your server:

1. Receives the HTTP request
2. Queries the database (`SELECT * FROM menu_items ...`)
3. Serializes the result to JSON
4. Sends the response

For a menu that changes once a day, you're hitting the database **thousands of times** for the exact same data.

```
Without cache:
User 1 → DB query → 45ms
User 2 → DB query → 45ms
User 3 → DB query → 45ms
... 1000 users → 1000 identical DB queries

With cache:
User 1 → DB query → 45ms → store in cache
User 2 → cache hit → 2ms
User 3 → cache hit → 2ms
... 1000 users → 1 DB query + 999 cache hits
```

When to Cache (and When NOT to)

Cache This	Don't Cache This
Menu items (changes rarely)	Shopping cart (changes per user per second)
Category list (almost static)	Real-time stock prices
Restaurant info (changes once a day)	Payment status (must be real-time)
Search results for popular queries	User-specific dashboards
Aggregated stats (daily revenue)	Data that must be instantly consistent

Rule of thumb: Cache data that is **read often** and **written rarely**.

Part 2: What is Redis?

Redis = Remote Dictionary Server

An in-memory key-value store. Think of it as a giant Python dictionary that lives outside your app, is insanely fast (100K+ operations/second), and can be shared across multiple servers.

```
Regular database (PostgreSQL):
Data lives on DISK → read = disk seek → ~5-45ms

Redis:
Data lives in RAM → read = memory lookup → ~0.1-2ms
```

Redis Data Types

Type	Example	Use Case
String	SET menu:items "[{...}]"	Cache API responses
Hash	HSET user:1 name "Prateek" email "p@x.com"	Cache object fields
List	LPUSE recent:orders "order_123"	Recent activity feeds
Set	SADD online:users "user_1" "user_2"	Track unique online users
Sorted Set	ZADD leaderboard 100 "user_1"	Rankings, leaderboards

For API caching, we mostly use **Strings** — store the entire JSON response as a string with a TTL.

TTL (Time To Live)

Every cached value has an expiry time. After TTL expires, Redis deletes it automatically.

```
SET menu:items "[{...}]" EX 300      # Expires in 300 seconds (5 minutes)

# After 5 minutes:
GET menu:items → (nil)                # Gone. Next request hits DB again.
```

Choosing TTL:

Data Type	Suggested TTL	Why
Menu items	5-15 minutes	Changes rarely, ok to be slightly stale
Category list	1 hour	Almost never changes
Search results	2-5 minutes	New items should appear reasonably fast
User session	30 minutes	Security — force re-auth
Payment status	Don't cache	Must be real-time

Part 3: Caching Strategies

1. Cache-Aside (Lazy Loading) — Most Common

```
Request comes in:
1. Check cache → HIT? Return cached data
2. MISS? Query database
3. Store result in cache (with TTL)
4. Return data
```

```
def get_menu_items():
    # 1. Check cache
    cached = cache.get('menu_items')
    if cached:
        return cached          # Cache HIT – fast!

    # 2. Cache MISS – query DB
    items = MenuItem.objects.all()
    data = serialize(items)

    # 3. Store in cache for 5 minutes
    cache.set('menu_items', data, timeout=300)

    return data
```

Pros: Simple, only caches what's actually requested **Cons:** First request is always slow (cache miss), possible stale data

2. Write-Through

When data changes:

1. Write to database
2. Immediately update cache

```
def update_menu_item(item_id, new_data):
    # Write to DB
    item = MenuItem.objects.get(id=item_id)
    item.name = new_data['name']
    item.save()

    # Immediately update cache
    cache.delete('menu_items') # Invalidate the list cache
```

Pros: Cache is always fresh **Cons:** Every write also writes to cache (slower writes)

3. Cache Invalidation

“There are only two hard things in Computer Science: cache invalidation and naming things.” — Phil Karlton

When the underlying data changes, you must either:

- **Delete the cache key** (next read will rebuild it)
- **Update the cache** (write-through)
- **Let it expire** (rely on TTL)

For our restaurant API, we use **delete on write + TTL as safety net**.

Part 4: Redis + Django — Implementation

Install

```
pip install django-redis redis
```

Configure in settings.py

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.redis.RedisCache',
        'LOCATION': 'redis://127.0.0.1:6379/1',
        'OPTIONS': {
            'CLIENT_CLASS': 'django_redis.client.DefaultClient',
        },
        'TIMEOUT': 300, # Default TTL: 5 minutes
    }
}
```

Three Levels of Caching in Django

Level 1: Per-View Cache (Easiest)

Cache the entire response of a view:

```
from django.utils.decorators import method_decorator
from django.views.decorators.cache import cache_page

class MenuItemListView(generics.ListAPIView):
    queryset = MenuItem.objects.select_related('category').all()
    serializer_class = MenuItemSerializer

    @method_decorator(cache_page(60 * 5)) # Cache for 5 minutes
    def list(self, request, *args, **kwargs):
        return super().list(request, *args, **kwargs)
```

Problem: `cache_page` caches the same response for ALL users. If the URL has query params (`?search=chicken&page=2`), each unique URL gets its own cache entry. That's actually good — different searches get different caches.

Level 2: Manual Cache (Most Control)

```
from django.core.cache import cache

class MenuItemListView(generics.ListAPIView):
    queryset = MenuItem.objects.select_related('category').all()
    serializer_class = MenuItemSerializer

    def list(self, request, *args, **kwargs):
        # Build a unique cache key from query params
        cache_key = f"menu_items:{request.GET.urlencode()}"

        cached_response = cache.get(cache_key)
        if cached_response:
            return Response(cached_response)

        # Cache miss — get from DB
        response = super().list(request, *args, **kwargs)

        # Cache the response data for 5 minutes
        cache.set(cache_key, response.data, timeout=300)
        return response
```

Level 3: Low-Level QuerySet Cache

```
from django.core.cache import cache

def get_all_categories():
    cache_key = 'all_categories'
    categories = cache.get(cache_key)

    if categories is None:
        categories = list(Category.objects.values('id', 'name'))
        cache.set(cache_key, categories, timeout=3600) # 1 hour

    return categories
```

Cache Invalidation in Practice

```
# In your admin or update view:
from django.core.cache import cache

def update_menu_item(request, item_id):
    # ... update the item in DB ...

    # Invalidate relevant caches
    cache.delete('menu_items:') # Clear the list cache
    cache.delete(f'menu_item:{item_id}') # Clear specific item cache

    # Or nuclear option: clear ALL cache
    # cache.clear()
```

Part 5: Measuring the Impact

Before Cache

```
# Hit the API 100 times, measure average response time
ab -n 100 -c 10 http://127.0.0.1:8000/menu/items/

# Or with curl:
time curl http://127.0.0.1:8000/menu/items/ > /dev/null
```

After Cache

```
# First request: cache MISS (hits DB)
time curl http://127.0.0.1:8000/menu/items/ > /dev/null
# → 45ms

# Second request: cache HIT
time curl http://127.0.0.1:8000/menu/items/ > /dev/null
# → 3ms
```

Verify in Redis CLI

```
redis-cli
> KEYS * # See all cached keys
> GET "menu_items:" # See cached data
> TTL "menu_items:" # See remaining TTL in seconds
> FLUSHDB # Clear all cache (nuclear option)
```

Part 6: Common Caching Pitfalls

Pitfall	What Happens	Fix
Cache stampede	Cache expires, 1000 users hit DB simultaneously	Use cache lock or staggered TTL
Stale data	Menu price updated but cache shows old price	Invalidate cache on write
Cache everything	Redis runs out of memory	Only cache hot paths, set maxmemory
No TTL	Cached data lives forever, never refreshes	Always set TTL
Cache per-user data	Redis fills up with 100K user caches	Only cache shared data
Ignoring cache misses	App crashes when Redis is down	Use try/except, fallback to DB

Part 7: Mentioning Your Project in Resume

The Wrong Way

Restaurant Management System

- Built a Django REST API
- Used PostgreSQL database
- Implemented CRUD operations
- Added pagination and search

This reads like a tutorial checklist. Every CS student has done this. Nothing stands out.

The Right Way – STAR Format with Technical Depth

Restaurant Order Management Platform (Django REST Framework, PostgreSQL, Redis, Razorpay)

- Designed and built REST APIs for menu management, order processing, and payment integration – handling 150+ menu items across 10 categories with paginated, searchable, and sortable endpoints
- Integrated Razorpay Payment Links API with webhook-based payment confirmation, signature verification, and idempotent event processing to prevent duplicate charges
- Built an automated reconciliation system (Django management command + cron) that compares local payment records with Razorpay's API to catch missed webhooks – resolving payment mismatches without manual intervention
- Implemented Redis caching (cache-aside pattern) on high-read endpoints, reducing average API response time from ~45ms to ~3ms for cached requests
- Prevented N+1 query problems using select_related/prefetch_related, reducing database queries per request from 21 to 1 on paginated list views

What Makes This Better?

Weak Version	Strong Version	Why It's Better
"Built a Django API"	"Designed REST APIs for menu management, order processing, and payment integration"	Specificity – tells WHAT the API does
"Used PostgreSQL"	"Prevented N+1 queries using select_related, reducing queries from 21 to 1"	Shows you understand performance
"Added		Shows you built production

pagination" Weak Version	"Paginated, searchable, and sortable endpoints handling 150+ items" Strong Version	features Why It's Better
"Integrated payments"	"Razorpay webhook-based confirmation with signature verification and idempotent processing"	Shows security and reliability awareness
"Added caching"	"Redis caching (cache-aside pattern) reducing response time from ~45ms to ~3ms"	Quantified impact + named the pattern

Key Resume Principles

1. **Lead with impact, not technology** — "Reduced response time by 93%" > "Used Redis"
2. **Name the patterns** — "cache-aside pattern", "idempotent processing", "reconciliation" show you understand WHY, not just HOW
3. **Quantify everything** — response times, query counts, item counts, percentage improvements
4. **Show architectural decisions** — "webhook over polling for real-time updates" shows you made deliberate choices
5. **Mention error handling** — "signature verification to prevent spoofed webhooks" shows security awareness

What Interviewers Will Ask About Each Bullet

Your Bullet	Their Follow-up
"Integrated Razorpay webhooks"	"What happens if a webhook fails? How do you prevent double-processing?"
"Built reconciliation cron"	"How often does it run? What edge cases did you handle?"
"Redis caching with cache-aside"	"How do you handle cache invalidation? What's your TTL strategy?"
"Prevented N+1 queries"	"What's the N+1 problem? How does select_related solve it?"
"Paginated endpoints"	"Offset vs cursor pagination — when would you use each?"

You can answer **ALL** of these because you built it. That's the point of this project — not just code, but interview ammunition.

Project Section Template

PROJECT NAME	[Tech Stack]
Brief one-line description	
• [Action verb] + [what you built] + [technical detail] + [measurable impact] • [Action verb] + [what you built] + [technical detail] + [measurable impact] • [Action verb] + [what you built] + [technical detail] + [measurable impact]	

Strong action verbs: Designed, Built, Implemented, Integrated, Optimized, Reduced, Automated, Architected, Migrated, Refactored

Weak action verbs: Used, Worked on, Helped with, Learned, Studied

Resources

- [Django Cache Framework Docs](#)
- [django-redis Documentation](#)
- [Redis Official Documentation](#)
- [Redis University — Free Courses](#)
- [Caching Strategies — AWS](#)
- [Circuit Breaker Pattern — Martin Fowler](#)

For the hands-on implementation guide, see [HANDS-ON-GUIDE.md](#)